

# Introducing Quail: An Environment for Agentic Qualitative Analysis

## Presenters

Dashiell Coyier <[coyierd@carleton.edu](mailto:coyierd@carleton.edu)>, Data Scientist for the DataSquad, Carleton College,  
0009-0003-1650-9370

## Keywords

Agentic AI, qualitative analysis, research software

# Abstract

Through our work on the Carleton College DataSquad [1], a student-staffed data and software support group, we have seen campus offices collect rich open-ended survey answers and review them superficially, for lack of time and infrastructure. Retrieval-augmented generation (RAG) [2] helps by giving a language model the passages that best match a question. That can show what students said about waiting for health center appointments, but not how often they said it or whether it varied by class year, since both depend on every entry.

Quail is an environment in which an AI agent analyzes a text dataset the way an analyst works in a Jupyter notebook [3]. With `quail exec`, the agent sends a cell of Python to a kernel that stays alive between calls and reads back its output. Cells use an analysis language, a Python library whose expressions select and rank entries using keyword scores (BM25 [4]), semantic scores, regular expressions and comparisons, and whose four verbs `retrieve`, `count`, `values` and `tag` act on them. For the survey, a cell might count answers about waiting by class year:

```
waits = Field("answer").semantic("waited weeks for an appointment") > 0.5
print(count(waits, by=Field("class_year")))
```

A later cell can read a random sample of those answers and, if they fit, label them with `tag(waits, "theme", "waiting")`. Quail thus builds on RAG, adding counts over the whole dataset and tags that record what the agent decided. Unlike Cloudflare's Code Mode [5], which also gives agents their tools as a code library, Quail keeps its kernel between cells.

Quail records each step so an analysis can be checked and continued. The tags a cell writes are saved to the session log, with the cell's code and output, before the agent sees the result. Tags survive kernel resets, and each can be traced to the code that produced it, which matters because an agent's labels can carry a model's biases. Cells run in a sandbox with no network, file or subprocess access, and the source data is never modified.

A study is a folder of text files, including the source CSVs and session logs, that git can share like code. Each kernel run writes its own log, so agents in separate sessions can push work without conflicts, and `quail fork` copies a session to try another coding scheme. Needing only a shell, Quail runs wherever an agent does, including cloud agents' temporary virtual machines [6, 7], which can start from a snapshot that already holds Quail and a prepared study. Quail 0.94 is an alpha release for Linux and macOS under the Apache 2.0 license [8]. Its results have not yet been formally evaluated, and working out how to do that well is our next step.

## References

1. The DataSquad Experiment: Lessons for Preparing Data and Computer Scientists for Work. Paula Lackie, Elliot Pickens, and Dashiell Coyier. arXiv:2511.19688. 2025. <https://arxiv.org/abs/2511.19688>
2. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Patrick Lewis et al. Advances in Neural Information Processing Systems 33. 2020. <https://arxiv.org/abs/2005.11401>
3. Jupyter Notebooks – a publishing format for reproducible computational workflows. Thomas Kluyver et al. Positioning and Power in Academic Publishing: Players, Agents and Agendas. IOS Press. 2016. <https://doi.org/10.3233/978-1-61499-649-1-87>
4. The Probabilistic Relevance Framework: BM25 and Beyond. Stephen Robertson and Hugo Zaragoza. Foundations and Trends in Information Retrieval. 2009. <https://doi.org/10.1561/15000000019>
5. Code Mode: the better way to use MCP. Kenton Varda and Sunil Pai. Cloudflare Blog. September 26, 2025. <https://blog.cloudflare.com/code-mode/>
6. Introducing Codex. OpenAI. May 16, 2025. <https://openai.com/index/introducing-codex/>
7. Cursor agents can now control their own computers. Jonas Nelle and Alexi Robbins. Cursor. February 24, 2026. <https://cursor.com/blog/agent-computer-use>
8. Quail. Dashiell Coyier. GitHub repository. 2026. <https://github.com/dcoyier/Quail>

## Connection to Mission, Goals, & Interests of US-RSE Community

This poster connects to US-RSE's Resources and Advocacy goals. For Resources, Quail is an open-source reference, under the Apache 2.0 license, for research software engineers building AI-assisted tools whose results researchers need to be able to check. The design decisions it presents come up in other projects as well: giving an agent a code interface to research data instead of many separate tool calls, keeping the source data read-only and running the agent's code in a sandbox, writing each step to a log before the agent sees its result, and storing a study as plain files that git can share and review. The poster gives the reasons for each decision, so that it is useful to people building their own tools as well as to people who might use Quail.

For Advocacy, Quail is an example of what a campus data and software support group can build in response to a local need. It came out of the DataSquad, a student-staffed group at Carleton College, after we saw campus offices collect open-ended survey answers they did not have time to analyze. It also addresses this year's theme, Advancing Science in the Age of AI, for qualitative research, where readers need to see how an analysis reached its conclusions before they can rely on it.

## AI Acknowledgement

OpenAI ChatGPT (GPT-5.6 Thinking) was used during the revision of the submitted abstract. For this camera-ready version, Anthropic Claude (Claude Opus 5.5, used through a Cursor cloud agent) drafted the revised Abstract, References, and Connection sections from the submitted abstract and the Quail documentation. The author reviewed and verified all content and takes responsibility for the submitted content.